

پایتون حرفه‌ای

از صفر تا سطح جونیور دولوپر

جزوه کامل، پروژه‌محور و قابل تدریس

مناسب نوجوانان ۱۳–۱۸ سال و بزرگسالان مبتدی تا نیمه‌حرفه‌ای

تدریس کلاسی

PEP ۸ • Python ۳٫۱۰+ • پروژه‌محور • سنجش‌پذیر

بخش ۰ و بخش ۱ — مبانی

AHRIVAN | اهریمن

بخش ۰ — مقدمه برای مدرس و خودآموز

۰٫۱ فلسفه دوره

این دوره بر اساس طراحی معکوس (Backward Design) ساخته شده است: اول مشخص کرده‌ایم مخاطب در پایان باید چه توانایی‌های قابل اندازه‌گیری داشته باشد، سپس محتوا، فعالیت‌ها و روش‌های سنجش را حول همان اهداف طراحی کرده‌ایم.

رویکرد اصلی، یادگیری پروژه‌محور (Project-Based Learning) است. هر مفهوم با یک انگیزه واقعی شروع می‌شود، با مثال صنعتی توضیح داده می‌شود، سپس کد نوشته می‌شود و در نهایت در پروژه به کار می‌رود.

هدف نهایی این نیست که فقط «سینتکس» یاد بگیرید. هدف این است که بتوانید مثل یک جونیور دولوپر فکر کنید: مسئله را بشکنید، کد تمیز بنویسید، خطا را بخوانید، تست بنویسید و پروژه را به نتیجه برسانید.

۰٫۲ پیش‌نیازها

- آشنایی پایه با کار با کامپیوتر (نصب نرم‌افزار، کار با فایل‌ها)
- توانایی تایپ فارسی و انگلیسی
- دسترسی به اینترنت برای نصب ابزارها و جستجوی مستندات
- هیچ پیش‌زمینه برنامه‌نویسی لازم نیست

۰٫۳ نقشه راه کلی دوره

- بخش ۱ — مبانی: نصب، متغیر، شرط، حلقه، تابع، ماژول
- بخش ۲ — ساختمان داده و کار با داده: لیست، دیکشنری، فایل، JSON، مدیریت خطا
- بخش ۳ — برنامه‌نویسی حرفه‌ای: PEP۸، توابع پیشرفته، OOP، تست، گیت
- بخش ۴ — پروژه‌های واقعی: To-Do، CLI، وب‌اسکرپر، پروژه پایانی
- بخش ۵ — سنجش و مسیر ادامه: آزمون‌ها، روبریک، مسیرهای بعدی

۰٫۴ روش مطالعه پیشنهادی (خودآموز)

- هر درس را یک‌جا تمام نکنید. ۲۰–۴۰ دقیقه مطالعه + کدنویسی فعال
- همه کدها را خودتان تایپ کنید، کپی نکنید
- تمرین پایه را حتماً حل کنید؛ تمرین چالشی را اگر وقت داشتید
- خطاها را عمداً ایجاد کنید تا پیام‌های واقعی را ببینید
- پروژه هر فصل را کامل انجام دهید؛ این مهم‌ترین بخش یادگیری است

۰٫۵ روش تدریس پیشنهادی (مدرس)

- جلسات ۹۰ دقیقه‌ای: ۲۰ دقیقه مفهوم + ۳۰ دقیقه کدنویسی زنده + ۳۰ دقیقه تمرین + ۱۰ دقیقه جمع‌بندی

- کد را زنده بنویسید و عمداً خطا ایجاد کنید تا دیباگ کردن را نشان دهید
- از تمایز آموزشی استفاده کنید: مسیر پایه برای همه، مسیر پیشرفته برای سریع‌ترها
- هر جلسه با یک سؤال سنجش تکوینی شروع یا تمام شود
- پروژه‌ها را در کلاس شروع کنید و در خانه تکمیل کنید

۰٫۶ ابزارهای مورد نیاز

- Python ۳٫۱۰ یا بالاتر (پیشنهاد: آخرین نسخه پایدار)
- VS Code + افزونه Python (مایکروسافت)
- ترمینال / PowerShell / Command Prompt
- Git (در بخش‌های پیشرفته)
- مرورگر برای مستندات رسمی: docs.python.org

۰٫۷ منابع تکمیلی

- مستندات رسمی پایتون: https://docs.python.org/۳/
- PEP ۸ — Style Guide: https://peps.python.org/pep-۰۰۰۸/
- Real Python (مقالات باکیفیت)
- Python Tutor (برای دیدن اجرای خط‌به‌خط کد)

این فایل شامل بخش ۰ و بخش ۱ (مبانی) است. بخش‌های بعدی به‌صورت جداگانه ادامه می‌یابند.

بخش ۱ — مبانی (Foundations)

نقشه فصل و اهداف کلی

در پایان این بخش، فراگیر باید بتواند محیط توسعه را راه‌اندازی کند، کدهای ساده تا متوسط بنویسد، از متغیر، شرط، حلقه و تابع به‌درستی استفاده کند و ماژول‌های استاندارد را import کند.

- درس ۱: نصب پایتون، VS Code و اولین اجرا
- درس ۲: print، کامنت و ورودی/خروجی
- درس ۳: متغیرها، انواع داده و تبدیل نوع
- درس ۴: عملگرها
- درس ۵: رشته‌ها و f-string
- درس ۶: ساختارهای شرطی
- درس ۷: حلقه‌ها
- درس ۸: توابع و پارامترها
- درس ۹: دامنه متغیرها (Scope)
- درس ۱۰: ماژول‌ها و import

درس ۱ — نصب پایتون، VS Code و اولین اجرا

۱. هدف یادگیری قابل اندازه‌گیری

در پایان این درس، فراگیر می‌تواند پایتون و VS Code را نصب کند، یک فایل .py بسازد، کد را اجرا کند و تفاوت بین اجرای اسکریپت و محیط تعاملی را توضیح دهد.

۲. پیش‌نیاز

آشنایی پایه با نصب نرم‌افزار روی سیستم‌عامل خود.

۳. زمان پیشنهادی

۴۵ تا ۶۰ دقیقه (شامل نصب)

۴. انگیزه / سناریوی واقعی

هر برنامه‌نویس حرفه‌ای اول محیط کارش را درست می‌کند. اگر محیط درست نباشد، بعداً ساعت‌ها وقت روی مشکلات نصب تلف می‌شود. این درس پایه‌ای‌ترین مهارت عملی شماست.

۵. واژه‌های کلیدی

- Interpreter (مفسر): برنامه‌ای که کد پایتون را اجرا می‌کند
- IDE / Editor: محیط نوشتن کد (اینجا VS Code)
- Script: فایل حاوی کد که یک‌جا اجرا می‌شود
- REPL / Interactive Mode: محیط تعاملی خط‌به‌خط
- Terminal: خط فرمان سیستم

۶. آموزش گام‌به‌گام

نصب پایتون

- به python.org بروید و آخرین نسخه پایدار را دانلود کنید.
- در ویندوز حتماً گزینه Add Python to PATH را فعال کنید.
- بعد از نصب، ترمینال را باز کنید و بنویسید: python --version
- باید نسخه‌ای مثل Python ۳٫۱۲٫x نمایش داده شود.

نصب VS Code

- از code.visualstudio.com دانلود و نصب کنید.
- افزونه Python (منتشرشده توسط Microsoft) را نصب کنید.
- یک پوشه برای پروژه‌های این دوره بسازید.

اولین برنامه

یک فایل به نام hello.py بسازید و کد زیر را داخلش بنویسید، سپس با دکمه Run یا دستور python hello.py اجرا کنید.

۷. کد نمونه کامل

```
# hello.py
print("سلام به دوره پایتون حرفه‌ای خوش آمدید!")
print("اولین قدم را برداشتید.")
```

۸. خروجی نمونه

```
سلام به دوره پایتون حرفه‌ای خوش آمدید!
اولین قدم را برداشتید.
```

۹. توضیح خط‌به‌خط

خط اول یک کامنت است و اجرا نمی‌شود. دو خط بعدی با print متن را در خروجی استاندارد چاپ می‌کنند.

۱۰. خطاهای رایج

خطا: 'python' is not recognized as an internal or external command

علت: پایتون به PATH اضافه نشده. راه‌حل: نصب مجدد با تیک Add to PATH یا اضافه کردن دستی مسیر.

خطا: SyntaxError: invalid syntax

علت: معمولاً پرانتز یا کوتیشن جا افتاده. راه‌حل: پرانتزها و کوتیشن‌های انگلیسی را بررسی کنید.

۱۱. تمرین پایه

فایلی به نام about_me.py بسازید که سه خط درباره خودتان چاپ کند.

۱۲. تمرین چالشی

در محیط تعاملی پایتون (دستور python در ترمینال) چند عبارت ریاضی ساده را امتحان کنید و نتیجه را مشاهده کنید.

۱۳. پاسخ تمرین‌ها

تمرین پایه — نمونه:

```
print("نام من آریا است.")
print("۱۷ سال دارم.")
print("می‌خواهم برنامه‌نویس شوم.")
```

تمرین چالشی: در ترمینال python را اجرا کنید، سپس مثلاً ۲ + ۳ یا ۱۰ * ۷ را بنویسید و Enter بزنید.

۱۴. مسیر تقویتی و مسیر پیشرفته

مسیر پایه: فقط نصب و اجرای یک فایل.

مسیر پیشرفته: با دستور python -i hello.py فایل را اجرا کنید و بعد از اجرا در محیط تعاملی بمانید.

۱۵. سنجش تکوینی

۱. دستور نمایش نسخه پایتون چیست؟

۲. فرق اجرای فایل .py با محیط تعاملی چیست؟

۳. چرا Add to PATH مهم است؟

۱۶. چک‌لیست

- ☐ پایتون را نصب کرده‌ام و python --version کار می‌کند
- ☐ VS Code را نصب و افزونه Python را فعال کرده‌ام
- ☐ یک فایل .py ساخته و اجرا کرده‌ام
- ☐ محیط تعاملی را باز کرده‌ام

درس ۲ — print، کامنت و ورودی/خروجی

۱. هدف یادگیری

فراگیر می‌تواند متن و مقدار متغیرها را چاپ کند، از کاربر ورودی بگیرد، کامنت بنویسد و تفاوت print و return را در سطح مقدماتی توضیح دهد.

۲. پیش‌نیاز

درس ۱ (محیط کار آماده باشد).

۳. زمان پیشنهادی

۳۵ تا ۴۵ دقیقه

۴. انگیزه

تقریباً هر برنامه‌ای باید با کاربر حرف بزند: چیزی نشان بدهد یا چیزی بپرسد. print و input پایه‌ای‌ترین ابزارهای ارتباط برنامه با دنیای بیرون هستند.

۵. واژه‌های کلیدی

- print — خروجی استاندارد
- input — ورودی از کاربر
- Comment — توضیح برای انسان
- stdout — جریان خروجی استاندارد

۶. آموزش گام‌به‌گام

print می‌تواند چند مقدار را با کاما از هم جدا کند. input همیشه رشته برمی‌گرداند. کامنت با # شروع می‌شود و تا پایان خط ادامه دارد.

۷. کد نمونه

```
#برنامه احوال‌پرسی ساده
name = input("نام شما چیست؟ ")
age = input("چند سال دارید؟ ")
print("سلام", name + "!")
print("پس شما", age, "ساله هستید.")
print("خوش آمدید به دنیای پایتون.")
```

۸. خروجی نمونه

```
نام شما چیست؟ سارا
چند سال دارید؟ ۱۶
سلام سارا!
```

```
پس شما ۱۶ ساله هستید.
خوش آمدید به دنیای پایتون.
```

۹. توضیح خط‌به‌خط

input پیام را نشان می‌دهد و منتظر Enter می‌ماند. مقدار واردشده همیشه str است. print با کاما بین آرگومان‌ها فاصله می‌گذارد.

۱۰. خطاهای رایج

پیام: NameError: name 'name' is not defined

علت: متغیر قبل از استفاده ساخته نشده یا اشتباه تایپ شده.

پیام: TypeError: can only concatenate str (not "int") to str

علت: تلاش برای جمع کردن رشته و عدد با +. راه‌حل: از کاما در print استفاده کنید یا str() کنید.

۱۱. تمرین پایه

برنامه‌ای بنویسید که شهر و رنگ مورد علاقه کاربر را بپرسد و در یک جمله اعلام کند.

۱۲. تمرین چالشی

برنامه‌ای بنویسید که دو عدد (به‌صورت متن) بگیرد و آن‌ها را کنار هم با علامت + چاپ کند (هنوز جمع ریاضی نکنید).

۱۳. پاسخ

```
city = input("شهرتان؟")
color = input("رنگ مورد علاقه؟")
print("شما اهل", city, "هستید و رنگ", color, "را دوست دارید.")
a = input("عدد اول: ")
b = input("عدد دوم: ")
print(a + " + " + b)
```

۱۴. تمایز آموزشی

مسیر پایه: فقط input و print.

مسیر پیشرفته: از sep و end در print استفاده کنید (مثلاً print(..., sep=' | ', end='\n\n')).

۱۵. سنجش تکوینی

۱. input چه نوعی برمی‌گرداند؟

۲. چطور چند مقدار را در یک print چاپ کنیم؟

۳. کامنت برای چه کسی نوشته می‌شود؟

۱۶. چک‌لیست

- ☐ می‌توانم از input استفاده کنم
- ☐ می‌توانم چند مقدار را با print چاپ کنم
- ☐ می‌توانم کامنت مفید بنویسم

درس ۳ — متغیرها، انواع داده و تبدیل نوع

۱. هدف یادگیری

فراگیر می‌تواند متغیر بسازد، نوع داده اصلی را تشخیص دهد، بین انواع تبدیل انجام دهد و توضیح دهد چرا نوع داده مهم است.

۲. پیش‌نیاز

درس ۲

۳. زمان پیشنهادی

۴۰ تا ۵۰ دقیقه

۴. انگیزه

در برنامه‌های واقعی، داده از کاربر، فایل یا اینترنت می‌آید و تقریباً همیشه نیاز به تبدیل نوع دارد. اشتباه در نوع داده یکی از رایج‌ترین منبع باگ‌هاست.

۵. واژه‌های کلیدی

- Variable — متغیر
- int, float, str, bool — انواع پایه
- type() — مشاهده نوع
- Casting — تبدیل نوع صریح

۶ و ۷. آموزش + کد نمونه

```
"آریاname: str = "
age: int = 17
height: float = 1.75
is_student: bool = True


print(type(name))    # <class 'str'>
print(type(age))     # <class 'int'>
print(type(height))  # <class 'float'>
print(type(is_student))  # <class 'bool'>


#تبدیل نوع
age_text = "17"
age_number = int(age_text)
print(age_number + 1)   # 18
```

۸. خروجی نمونه

```
<class 'str'>
```

```
<class 'int'>
<class 'float'>
<class 'bool'>
18
```

۹. توضیح

Type hint (مثل name: str) به پایتون نمی‌گوید نوع را اجبار کند، اما خوانایی و ابزارهای کمکی را بهتر می‌کند. int() و float() و str() تبدیل صریح انجام می‌دهند.

۱۰. خطاهای رایج

```
")۱۷>>> int("
'۱۷ValueError: invalid literal for int() with base 10: '
```

علت: ارقام فارسی. راه‌حل: از ارقام انگلیسی استفاده کنید.

```
>>> int("3.14")
ValueError: invalid literal for int() with base 10: '3.14'
```

راه‌حل: اول float کنید بعد int، یا از round استفاده کنید.

۱۱. تمرین پایه

متغیرهایی برای نام، سن، معدل و وضعیت دانش‌آموزی بسازید و نوع هرکدام را چاپ کنید.

۱۲. تمرین چالشی

دو عدد از کاربر بگیرید، آن‌ها را به float تبدیل کنید، جمع و میانگین را محاسبه و چاپ کنید.

۱۳. پاسخ

```
"سارا= name
age = 16
gpa = 18.5
is_student = True
print(type(name), type(age), type(gpa), type(is_student))
: "))عدد اول(a = float(input("
: "))عدد دوم(b = float(input("
:", a + b)جمع(print("
:", (a + b) / 2)میانگین(print("
```

۱۴. تمایز

مسیر پایه: فقط int و str و float.

مسیر پیشرفته: از type hint استفاده کنید و با isinstance() نوع را بررسی کنید.

۱۵. سنجش

۱. نوع برگشتی input چیست؟

۲. چطور رشته «۲۵» را به عدد تبدیل کنیم؟

۳. bool چه مقادیری دارد؟

۱۶. چک‌لیست

- ☐ می‌توانم چهار نوع پایه را بسازم و تشخیص دهم
- ☐ می‌توانم با int/float/str تبدیل انجام دهم
- ☐ پیام ValueError را می‌شناسم

درس ۴ — عملگرها

۱. هدف یادگیری

فراگیر می‌تواند از عملگرهای حسابی، مقایسه‌ای و منطقی به‌درستی استفاده کند و اولویت عملگرها را رعایت کند.

۲. پیش‌نیاز

درس ۳

۳. زمان

۳۵ دقیقه

۴. انگیزه

تقریباً هر منطق تجاری یا بازی یا محاسبه به عملگرها وابسته است: مقایسه قیمت، بررسی شرط ورود، محاسبه تخفیف و غیره.

۵. واژه‌های کلیدی

- Arithmetic operators: + - * / // % **
- Comparison: == != < > <= >=
- Logical: and, or, not
- Precedence — اولویت

۷. کد نمونه

```
a, b = 17, 5
print(a + b)   # 22
print(a - b)   # 12
print(a * b)   # 85
print(a / b)   # 3.4
print(a // b)  # 3 تقسیم صحیح
print(a % b)   # 2 باقی‌مانده
print(a ** 2)  # 289 توان

print(a > b)          # True
print(a == 17)        # True
print(a > 10 and b < 10)  # True
print(not (a < b))    # True
```

۸. خروجی

```
22
12
```

```
85
3.4
3
2
289
True
True
True
True
```

۱۰. خطاهای رایج

استفاده از = به جای == در شرط ⟵ منطق اشتباه یا SyntaxError در برخی ساختارها.

تقسیم بر صفر ⟵ ZeroDivisionError.

۱۱. تمرین پایه

برنامه‌ای بنویسید که دو عدد بگیرد و همه عملگرهای حسابی را روی آن‌ها اعمال و چاپ کند.

۱۲. تمرین چالشی

بررسی کنید آیا یک عدد هم زوج است و هم بزرگ‌تر از ۵۰؛ نتیجه را True/False چاپ کنید.

۱۳. پاسخ چالشی

```
: "))عددn = int(input("
print(n % 2 == 0 and n > 50)
```

۱۶. چک‌لیست

- ☐ فرق / و // را می‌دانم
- ☐ می‌توانم شرط ترکیبی با and/or بنویسم
- ☐ اولویت عملگرها را رعایت می‌کنم

درس ۵ — رشته‌ها و f-string

۱. هدف یادگیری

فراگیر می‌تواند رشته بسازد، برش دهد، متدهای رایج را به کار ببرد و با f-string خروجی خوانا تولید کند.

۲. پیش‌نیاز

درس ۳ و ۴

۳. زمان

۴۰ دقیقه

۴. انگیزه

در برنامه‌های واقعی بخش بزرگی از کار، ساختن پیام، گزارش، لاگ و رابط کاربری متنی است. f-string از Python ۳٫۶ به بعد استاندارد طلایی فرمت‌بندی است.

۵. واژه‌های کلیدی

- str methods: upper, lower, strip, replace, split, join
- Slicing — برش
- f-string — فرمت‌بندی مدرن
- Immutable — تغییرناپذیر

۷. کد نمونه

```
text = "  Python Programming  "
print(text.strip())
print(text.upper())
print(text.lower())
print(text.replace("Python", "Java"))
print(len(text))
print(text[2:8])

"سارا"name = "
score = 19.5
: {score:.1f}")نمره: {name} | نامprint(f"
.")است {score} عزیز، نمره شما {name}print(f"
```

۸. خروجی

```
Python Programming
```

```
PYTHON PROGRAMMING
python programming
  Java Programming
                22
            Python
نام :سارا | نمره19.5 :
سارا عزیز، نمره شما 19.5 است.
```

۱۰. خطاهای رایج

تلاش برای تغییر کاراکتر رشته با ایندکس ← TypeError (رشته تغییرناپذیر است).

فراموش کردن f قبل از رشته ← آکولادها به‌صورت خام چاپ می‌شوند.

۱۱. تمرین پایه

نام و نام خانوادگی را بگیرید، آن‌ها را strip و title کنید و با f-string خوش‌آمد بگویید.

۱۲. تمرین چالشی

جمله‌ای از کاربر بگیرید و تعداد کلمات آن را (با split) چاپ کنید.

۱۳. پاسخ

```
: ").strip().title()نامfirst = input("
: ").strip().title()نام خانوادگیlast = input("
 {first} {last}!")سلامprint(f"
: ").strip()جملهsentence = input("
words = sentence.split()
: {len(words)}")تعداد کلماتprint(f"
```

۱۶. چک‌لیست

- ☐ می‌توانم از متدهای strip/upper/replace استفاده کنم
- ☐ می‌توانم برش بزنم
- ☐ می‌توانم f-string بنویسم

درس ۶ — ساختارهای شرطی (if / elif / else)

۱. هدف یادگیری

فراگیر می‌تواند شرایط ساده و تودرتو بنویسد، از elif استفاده کند و منطق تصمیم‌گیری را در کد پیاده کند.

۲. پیش‌نیاز

درس ۴ (عملگرهای مقایسه‌ای و منطقی)

۳. زمان

۴۵ دقیقه

۴. انگیزه

تقریباً هر سیستم واقعی تصمیم می‌گیرد: آیا کاربر مجاز است؟ آیا موجودی کافی است؟ آیا نمره قبول است؟ شرط‌ها قلب منطق برنامه هستند.

۵. واژه‌های کلیدی

- if / elif / else
- Truthy / Falsy
- Nested conditions — شرط تودرتو
- Indentation — تورفتگی

۷. کد نمونه

```
: "))نمره را وارد کنید(score = float(input("


if score >= 18:
"عالی"    level = "
elif score >= 15:
"خوب"    level = "
elif score >= 10:
"قبول"    level = "
else:
"نیاز به تلاش بیشتر"    level = "


: {level}")وضعیت شما(print(f"


#مثال صنعتی ساده :کنترل دسترسی
 (admin/user/guest): ").strip().lower()نقش(role = input("
if role == "admin":
")دسترسی کامل    print("
elif role == "user":
```

```
")دسترسی محدود    print("
else:
")فقط مشاهده    print("
```

۱۰. خطاهای رایج

```
  File "main.py", line 3
    print("ok")
    ^^^^^
IndentationError: expected an indented block after 'if' statement
```

راه‌حل: بعد از : حتماً تورفتگی (۴ فاصله یا یک Tab) بدهید.

اشتباه رایج دیگر: نوشتن else if به جای elif.

۱۱. تمرین پایه

سن کاربر را بگیرید و بگویید کودک / نوجوان / بزرگسال است.

۱۲. تمرین چالشی

یک ماشین‌حساب ساده با منو بسازید: ۱-جمع ۲-تفریق ۳-ضرب ۴-تقسیم. بر اساس انتخاب کاربر عمل کنید و تقسیم بر صفر را مدیریت کنید.

۱۳. پاسخ پایه

```
: "))سنage = int(input("
if age < 13:
")کودک    print("
elif age < 18:
")نوجوان    print("
else:
")بزرگسال    print("
```

۱۶. چک‌لیست

- ☐ می‌توانم if/elif/else بنویسم
- ☐ تورفتگی را رعایت می‌کنم
- ☐ می‌توانم شرط ترکیبی بنویسم

درس ۷ — حلقه‌ها (for و while)

۱. هدف یادگیری

فراگیر می‌تواند با for و while تکرار ایجاد کند، از range استفاده کند و حلقه‌های بی‌نهایت را تشخیص و کنترل کند.

۲. پیش‌نیاز

درس ۶

۳. زمان

۵۰ دقیقه

۴. انگیزه

پردازش لیست کاربران، خواندن خطوط فایل، تلاش مجدد برای اتصال به سرور، بازی‌ها — همه به حلقه نیاز دارند.

۵. واژه‌های کلیدی

- for / while
- range(start, stop, step)
- break / continue
- Infinite loop — حلقه بی‌نهایت

۷. کد نمونه

```
 rangeبا# for
for i in range(1, 6):
 {i}")شماره    print(f"


# while
count = 3
while count > 0:
    print(count)
    count -= 1
!")شروعprint("


#مثال واقعی :منوی تکرارشونده
while True:
"): ").strip().lower() برای خروج (quit دستور    cmd = input("
    if cmd == "quit":
")خداحافظ        print("
        break
: {cmd}")دستور شما    print(f"
```

۱۰. خطاهای رایج

فراموش کردن به‌روزرسانی متغیر در while ← حلقه بی‌نهایت.

استفاده از range(len(list)) وقتی می‌توان مستقیم روی لیست پیمایش کرد (در بخش بعد بهتر می‌شود).

۱۱. تمرین پایه

اعداد زوج بین ۱ تا ۲۰ را با حلقه چاپ کنید.

۱۲. تمرین چالشی

بازی حدس عدد: کامپیوتر عددی بین ۱ تا ۵۰ انتخاب کند، کاربر حدس بزند، راهنمایی بزرگ‌تر/کوچک‌تر بدهد و تعداد تلاش را بشمارد.

۱۳. پاسخ پایه

```
for n in range(2, 21, 2):
    print(n)
```

۱۶. چک‌لیست

- ☐ می‌توانم for با range بنویسم
- ☐ می‌توانم while کنترل‌شده بنویسم
- ☐ می‌دانم break چه می‌کند

درس ۸ — توابع و پارامترها

۱. هدف یادگیری

فراگیر می‌تواند تابع تعریف کند، پارامتر و مقدار پیش‌فرض بدهد، مقدار برگرداند و توضیح دهد چرا تابع باعث کاهش تکرار و افزایش خوانایی می‌شود.

۲. پیش‌نیاز

درس‌های قبلی

۳. زمان

۵۰ دقیقه

۴. انگیزه

در کدهای واقعی، تکرار ممنوع است (اصل DRY). تابع یعنی بسته‌بندی یک مسئولیت مشخص. هرچه پروژه بزرگ‌تر شود، بدون تابع غرق می‌شوید.

۵. واژه‌های کلیدی

- def — تعریف تابع
- Parameter / Argument
- return
- Default argument
- Docstring

۷. کد نمونه

```
") -> str:سلامdef greet(name: str, greeting: str = "
."""  یک پیام خوش‌آمدگویی می‌سازد"""
    return f"{greeting} {name}!"


def calculate_average(a: float, b: float, c: float) -> float:
    return (a + b + c) / 3


"))سارا(print(greet("
"))درود", "آریا(print(greet("
avg = calculate_average(18, 19.5, 17)
: {avg:.2f}")میانگین(print(f"
```

۸. خروجی

```
سلام سارا!
درود آریا!
میانگین18.17 :
```

۱۰. خطاهای رایج

فراموش کردن return → تابع None برمی‌گرداند.

استفاده از متغیر بیرون تابع بدون global (در درس بعد عمیق‌تر).

۱۱. تمرین پایه

تابعی بنویسید که دو عدد بگیرد و بزرگ‌تر را برگرداند.

۱۲. تمرین چالشی

تابعی بنویسید که نمره بگیرد و رشته وضعیت (عالی/خوب/قبول/مردود) را برگرداند. سپس در حلقه از کاربر چند نمره بگیرید و وضعیت هرکدام را چاپ کنید.

۱۳. پاسخ پایه

```
def maximum(a: float, b: float) -> float:
    if a >= b:
        return a
    return b


print(maximum(10, 7))
```

۱۶. چک‌لیست

- ☐ می‌توانم تابع با پارامتر و return بنویسم
- ☐ می‌توانم مقدار پیش‌فرض تعریف کنم
- ☐ docstring ساده می‌نویسم

درس ۹ — دامنه متغیرها (Scope)

۱. هدف یادگیری

فراگیر می‌تواند تفاوت متغیر محلی و سراسری را توضیح دهد و از سایه انداختن (shadowing) و استفاده نادرست از global اجتناب کند.

۲. پیش‌نیاز

درس ۸

۳. زمان

۳۰ دقیقه

۴. انگیزه

باگ‌های عجیب خیلی وقت‌ها از اشتباه در scope می‌آیند. درک دامنه، پیش‌نیاز نوشتن توابع و کلاس‌های تمیز است.

۵. واژه‌های کلیدی

- Local scope — دامنه محلی
- Global scope — دامنه سراسری
- LEGB rule
- global / nonlocal

۷. کد نمونه

```
سراسریcounter = 0  #

def increment():
    global counter
    counter += 1

def local_example():
محلی    x = 10  #
:", x)داخل تابع    print("

increment()
increment()
print("counter =", counter)
local_example()
# print(x)  # NameError
```

۱۰. خطاهای رایج

```
UnboundLocalError: local variable 'counter' referenced before assignment
```

علت: داخل تابع به متغیر نسبت داده شده و پایتون آن را محلی فرض کرده، اما قبل از انتساب خوانده شده. راه‌حل: global یا بهتر از آن، بازگرداندن مقدار جدید.

- در کد حرفه‌ای تا جای ممکن از global اجتناب کنید. بهتر است مقدار را برگردانید و در بیرون نسبت دهید.

۱۱. تمرین پایه

تابعی بنویسید که یک عدد سراسری را دو برابر کند (با global) و نتیجه را نشان دهد.

۱۲. تمرین چالشی

همان کار را بدون global و فقط با return انجام دهید.

۱۶. چک‌لیست

- فرق محلی و سراسری را می‌دانم
- می‌دانم چرا global معمولاً توصیه نمی‌شود

درس ۱۰ — ماژول‌ها و import

۱. هدف یادگیری

فراگیر می‌تواند ماژول‌های استاندارد را import کند، از توابع آن‌ها استفاده کند و ساختار import را مطابق عرف بنویسد.

۲. پیش‌نیاز

درس ۸

۳. زمان

۴۰ دقیقه

۴. انگیزه

چرخ را از نو اختراع نکنید. کتابخانه استاندارد پایتون بسیار غنی است: تاریخ، تصادفی، ریاضی، فایل، سیستم‌عامل و ...

۵. واژه‌های کلیدی

- import / from ... import
- Standard library
- random, math, datetime
- alias — as

۷. کد نمونه

```
import random
import math
from datetime import datetime


print(random.randint(1, 100))
print(math.sqrt(16))
print(math.pi)
print(datetime.now().strftime("%Y-%m-%d %H:%M"))


#انتخاب تصادفی از لیست
"]سبز", "آبی", "قرمزcolors = ["
print(random.choice(colors))
```

۱۰. خطاهای رایج

ModuleNotFoundError: ماژول نصب نیست یا نام اشتباه است.

استفاده از from random import * (توصیه نمی‌شود چون namespace را آلوده می‌کند).

۱۱. تمرین پایه

با random یک عدد بین ۱ تا ۶ (شبیه‌سازی تاس) چاپ کنید.

۱۲. تمرین چالشی

برنامه‌ای بنویسید که تاریخ و ساعت فعلی را به‌صورت خوانا چاپ کند و یک رمز عبور تصادفی ۸ کاراکتری از حروف و ارقام بسازد.

۱۳. پاسخ پایه

```
import random
print(random.randint(1, 6))
```

۱۶. چک‌لیست

- ☐ می‌توانم import و from import بنویسم
- ☐ از random و math استفاده کرده‌ام
- ☐ می‌دانم چرا import * توصیه نمی‌شود

پروژه فصل ۱ — ماشین‌حساب خط فرمان (CLI Calculator)

صورت مسئله

یک ماشین‌حساب متنی بنویسید که در حلقه اجرا شود، از کاربر عمل (+ - * /) و دو عدد را بگیرد، نتیجه را نمایش دهد و با دستور exit خارج شود. تقسیم بر صفر را مدیریت کند.

راهنمای گام‌به‌گام

- ۱. یک تابع برای نمایش منو
- ۲. یک تابع برای انجام محاسبه
- ۳. حلقه while True برای تکرار
- ۴. بررسی ورودی و مدیریت خطا

کد نمونه کامل

```
def calculate(op: str, a: float, b: float) -> float | None:
    if op == "+":
        return a + b
    if op == "-":
        return a - b
    if op == "*":
        return a * b
    if op == "/":
        if b == 0:
.")خطا :تقسیم بر صفر مجاز نیست            print("
            return None
        return a / b
")عملگر نامعتبر    print("
    return None


def main() -> None:
 ===")ماشین‌حساب ساده    print("===
    while True:
 exit: ").strip()یا (+ - * /) عمل        op = input("
        if op.lower() == "exit":
!")خداحافظ            print("
            break
        try:
: "))عدد اول            a = float(input("
: "))عدد دوم            b = float(input("
        except ValueError:
.")ورودی عددی معتبر نیست            print("
            continue
        result = calculate(op, a, b)
        if result is not None:
: {result}")نتیجه            print(f"


if __name__ == "__main__":
    main()
```

روبریک ارزیابی پروژه

- اجرا بدون خطا (۲۵٪)
- مدیریت تقسیم بر صفر و ورودی غیرعددی (۲۵٪)
- استفاده از تابع و ساختار تمیز (۲۵٪)
- خوانایی و رعایت نام‌گذاری (۲۵٪)

آزمون پایان بخش ۱ (۱۰ سؤال)

۱. دستور نمایش نسخه پایتون چیست؟

۲. نوع برگشتی input چیست؟

۳. فرق / و // چیست؟

۴. خروجی عبارت ۳ ** ۲ چیست؟

۵. چرا بعد از if باید تورفتگی بدهیم؟

۶. چطور حلقه while را متوقف کنیم؟

۷. کلمه کلیدی تعریف تابع چیست؟

۸. اگر return ننویسیم تابع چه برمی‌گرداند؟

۹. import random چه کاری انجام می‌دهد؟

۱۰. یک مزیت اصلی استفاده از تابع را بنویسید.

پاسخنامه

۱. python --version

۲. str

۳. / تقسیم اعشاری، // تقسیم صحیح

۴. ۹

۵. پایتون بلوک کد را با تورفتگی مشخص می‌کند

۶. با break یا غلط شدن شرط

۷. def

۸. None

۹. ماژول random را در دسترس قرار می‌دهد

۱۰. کاهش تکرار، خوانایی، قابلیت استفاده مجدد

خلاصه بخش ۱ و مهارت نهایی

خلاصه

در این بخش محیط کار را راه‌اندازی کردید، با متغیرها و انواع داده کار کردید، شرط و حلقه نوشتید، تابع ساختید و با ماژول‌های استاندارد آشنا شدید.

همچنین اولین پروژه CLI واقعی را پیاده‌سازی کردید.

مهارت نهایی مورد انتظار در پایان بخش ۱

- ☐ می‌توانم محیط پایتون و VS Code را راه‌اندازی کنم
- ☐ می‌توانم ورودی بگیرم، خروجی مناسب بسازم و نوع داده را مدیریت کنم
- ☐ می‌توانم منطق شرطی و حلقه‌ای بنویسم
- ☐ می‌توانم تابع تعریف کنم و از ماژول‌های استاندارد استفاده کنم
- ☐ می‌توانم یک برنامه کوچک CLI با مدیریت خطای پایه بنویسم

☐ بخش ۱ به پایان رسید. شما الان پایه محکمی دارید و آماده‌اید وارد ساختمان داده‌ها و کار حرفه‌ای با داده شوید.

☐ برای دریافت بخش ۲ (ساختمان داده، فایل، JSON و مدیریت خطا) بگویید: ادامه بده

AHRIVAN | اهریمن



بخش ۲

ساختمان داده و کار با داده

لیست • تاپل • ست • دیکشنری • کامپرهنشن
فایل متنی • JSON • مدیریت خطا

پایتون حرفه‌ای — از صفر تا جونیور دولوپر

AHRIVAN | اهریمن

نقشه بخش ۲ و اهداف کلی

در پایان این بخش، فراگیر باید بتواند ساختمان‌داده‌های اصلی پایتون را انتخاب و به‌کار ببرد، داده را از فایل و JSON بخواند و بنویسد، و خطاها را به‌صورت حرفه‌ای مدیریت کند.

- درس ۱۱: لیست‌ها (List)
- درس ۱۲: تاپل و ست (Tuple & Set)
- درس ۱۳: دیکشنری‌ها (Dictionary)
- درس ۱۴: کامپرهنشن و پیمایش پیشرفته
- درس ۱۵: کار با فایل متنی
- درس ۱۶: JSON
- درس ۱۷: مدیریت خطا با try / except
- پروژه فصل: مدیر وظایف (To-Do) با ذخیره JSON

درس ۱۱ — لیست‌ها (List)

۱. هدف یادگیری قابل اندازه‌گیری

فراگیر می‌تواند لیست بسازد، ایندکس‌گذاری و برش انجام دهد، متدهای اصلی را به کار ببرد و توضیح دهد چه زمانی لیست انتخاب مناسبی است.

۲. پیش‌نیاز

بخش ۱ (متغیر، حلقه، تابع)

۳. زمان پیشنهادی

۴۵–۵۰ دقیقه

۴. انگیزه / سناریوی واقعی

تقریباً هر برنامه‌ای با مجموعه‌ای از آیتم‌ها سروکار دارد: لیست کارها، سبد خرید، نتایج جستجو، نمرات کلاس. لیست پرکاربردترین ساختمان‌داده در پایتون است.

۵. واژه‌های کلیدی

- List — لیست (مرتب و تغییرپذیر)
- Index — ایندکس (از صفر)
- Slicing — برش
- append / insert / extend / remove / pop / sort

۷. کد نمونه کامل

```
"]ورزش", "کدنویسی", "مطالعه"tasks: list[str] = ["
مطالعهprint(tasks[0])          #
ورزشprint(tasks[-1])          #
']کدنویسی', 'مطالعهprint(tasks[0:2])        # ['


")خریدtasks.append("
")مرور درسtasks.insert(1, "
print(tasks)


")ورزشtasks.remove("
last = tasks.pop()
:", last)حذف‌شدهprint("
:", tasks)باقی‌ماندهprint("


numbers = [5, 2, 9, 1]
numbers.sort()
```

```
print(numbers)           # [1, 2, 5, 9]
print(sorted(numbers, reverse=True))
```

۸. خروجی نمونه

```
مطالعه
ورزش
']مطالعه', 'کدنویسی['
']مطالعه', 'مرور درس', 'کدنویسی', 'ورزش', 'خرید['
حذف‌شده: خرید
باقی‌مانده: ['مطالعه', 'مرور درس', 'کدنویسی']
[1, 2, 5, 9]
[9, 5, 2, 1]
```

۹. توضیح خط‌به‌خط

لیست با [] ساخته می‌شود. ایندکس منفی از انتها می‌شمارد. append به آخر اضافه می‌کند، insert در جایگاه مشخص. pop آخرین عنصر را برمی‌گرداند و حذف می‌کند. sort لیست را درجا مرتب می‌کند؛ sorted نسخه جدید می‌سازد.

۱۰. خطاهای رایج

```
IndexError: list index out of range
```

علت: دسترسی به ایندکسی که وجود ندارد. راه‌حل: قبل از دسترسی طول را بررسی کنید یا از try استفاده کنید.

```
ValueError: list.remove(x): x not in list
```

قبل از remove با in بررسی کنید.

۱۱. تمرین پایه

لیستی از ۵ میوه بسازید، یکی اضافه کنید، یکی حذف کنید و لیست نهایی را چاپ کنید.

۱۲. تمرین چالشی

از کاربر چند عدد بگیرید (تا وقتی که بنویسد stop)، آن‌ها را در لیست بریزید و میانگین و بزرگ‌ترین را چاپ کنید.

۱۳. پاسخ پایه

```
"]کیوی", "انگور", "پرتقال", "موز", "سیبfruits = ["
")انارfruits.append("
")موزfruits.remove("
print(fruits)
```

۱۴. تمایز آموزشی

مسیر پایه: append و remove و پیمایش با for.

مسیر پیشرفته: از enumerate و list comprehension ساده استفاده کنید.

۱۵. سنجش تکوینی

۱. فرق append و insert چیست؟

۲. tasks[-۱] چه چیزی برمی‌گرداند؟

۳. چطور لیست را بدون تغییر نسخه اصلی مرتب کنیم؟

۱۶. چک‌لیست

- ☐ می‌توانم لیست بسازم و ایندکس‌گذاری کنم
- ☐ متدهای append/remove/pop/sort را بلدم
- ☐ IndexError را می‌شناسم و مدیریت می‌کنم

درس ۱۲ — تاپل و ست (Tuple & Set)

۱. هدف یادگیری

فراگیر می‌تواند تاپل و ست بسازد، تفاوت آن‌ها با لیست را توضیح دهد و در موقعیت مناسب از هرکدام استفاده کند.

۲. پیش‌نیاز

درس ۱۱

۳. زمان

۴۰ دقیقه

۴. انگیزه

گاهی نمی‌خواهیم داده تغییر کند (مختصات، تنظیمات ثابت) ⟵ تاپل. گاهی فقط عضویت یکتا مهم است (تگ‌ها، شناسه‌ها) ⟵ ست. انتخاب درست ساختمان‌داده بخشی از تفکر مهندسی است.

۵. واژه‌های کلیدی

- Tuple — تاپل (مرتب و تغییرناپذیر)
- Set — مجموعه (نامرتب و یکتا)
- Immutable / Mutable
- union / intersection / difference

۷. کد نمونه

```
#تاپل
point: tuple[int, int] = (10, 20)
print(point[0])
x, y = point          # unpacking
print(f"x={x}, y={y}")


#ست
tags: set[str] = {"python", "backend", "api"}
tags.add("django")
tags.add("python")    #تکراری اضافه نمی‌شود
print(tags)


a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)          #اجتماع
print(a & b)          #اشتراک
print(a - b)          #تفاضل
```

۸. خروجی

```
10
x=10, y=20
{'python', 'backend', 'api', 'django'}
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
```

۱۰. خطاهای رایج

```
TypeError: 'tuple' object does not support item assignment
```

تاپل تغییرناپذیر است. اگر نیاز به تغییر دارید از لیست استفاده کنید.

۱۱. تمرین پایه

یک تاپل برای اطلاعات یک کتاب (عنوان، نویسنده، سال) بسازید و با unpacking چاپ کنید.

۱۲. تمرین چالشی

دو لیست از علاقه‌مندی‌های دو نفر بگیرید، آن‌ها را به ست تبدیل کنید و اشتراک و تفاضل را نشان دهید.

۱۶. چک‌لیست

- [ ] فرق لیست و تاپل را می‌دانم
- [ ] می‌توانم ست بسازم و عملیات مجموعه‌ای انجام دهم
- [ ] می‌دانم چه زمانی از کدام استفاده کنم

درس ۱۳ — دیکشنری‌ها (Dictionary)

۱. هدف یادگیری

فراگیر می‌تواند دیکشنری بسازد، مقدار را با کلید بخواند و به‌روز کند، روی کلید/مقدار پیمایش کند و کاربرد واقعی آن را پیاده کند.

۲. پیش‌نیاز

درس ۱۱

۳. زمان

۴۵ دقیقه

۴. انگیزه

دیکشنری مدل طبیعی بسیاری از داده‌های واقعی است: پروفایل کاربر، تنظیمات، رکورد دیتابیس، پاسخ API. در پروژه‌های واقعی بدون دیکشنری تقریباً غیرممکن است.

۵. واژه‌های کلیدی

- Dict — دیکشنری (کلید → مقدار)
- Key / Value
- items() / values() / keys()
- get — دسترسی امن

۷. کد نمونه

```
user: dict[str, str | int] = {
",سارا    "name": "
    "age": 17,
"تهران    "city": "
}
print(user["name"])
امن"))  # ندارد print(user.get("email", "

user["age"] = 18
user["email"] = "sara@example.com"

for key, value in user.items():
    print(f"{key}: {value}")

 JSON) رایج در تو در تو) #دیکشنری
product = {
```

```
    "id": 101,
",کتاب پایتون    "title": "
    "price": 250000,
"]برنامه‌نویسی", "آموزشی    "tags": ["
}
print(product["tags"][0])
```

۸. خروجی

```
سارا
ندارد
name: سارا
age: 18
city: تهران
email: sara@example.com
آموزشی
```

۱۰. خطاهای رایج

```
KeyError: 'email'
```

کلید وجود ندارد. راه‌حل: از get با مقدار پیش‌فرض استفاده کنید یا قبلش با in بررسی کنید.

۱۱. تمرین پایه

دیکشنری برای یک دانش‌آموز (نام، کلاس، معدل) بسازید و همه آیتم‌ها را چاپ کنید.

۱۲. تمرین چالشی

دفترچه تلفن ساده: چند نام و شماره بگیرید و در دیکشنری ذخیره کنید، سپس امکان جستجو بر اساس نام داشته باشید.

۱۶. چک‌لیست

- ☐ می‌توانم دیکشنری بسازم و به‌روز کنم
- ☐ از get و items استفاده می‌کنم
- ☐ KeyError را می‌شناسم

درس ۱۴ — کامپرهنشن و پیمایش پیشرفته

۱. هدف یادگیری

فراگیر می‌تواند list/dict comprehension بنویسد و کدهای تکراری ساده را کوتاه‌تر و خواناتر کند.

۲. پیش‌نیاز

درس ۱۱ و ۱۳

۳. زمان

۴۰ دقیقه

۴. انگیزه

در کد حرفه‌ای، comprehension برای تبدیل و فیلتر کردن داده‌ها بسیار رایج است و نشانه تسلط نسبی به پایتون محسوب می‌شود.

۵. واژه‌های کلیدی

- List Comprehension
- Dict Comprehension
- enumerate / zip

۷. کد نمونه

```
numbers = [1, 2, 3, 4, 5, 6]
squares = [n ** 2 for n in numbers]
evens = [n for n in numbers if n % 2 == 0]
print(squares)
print(evens)

names = ["sara", "arya", "reza"]
lengths = {name: len(name) for name in names}
print(lengths)

for index, name in enumerate(names, start=1):
    print(f"{index}. {name}")

scores = [18, 19, 17]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

۸. خروجی

```
[1, 4, 9, 16, 25, 36]
```

```
[2, 4, 6]
{'sara': 4, 'arya': 4, 'reza': 4}
1. sara
2. arya
3. reza
sara: 18
arya: 19
reza: 17
```

۱۱. تمرین پایه

با comprehension لیستی از مربع اعداد ۱ تا ۱۰ بسازید.

۱۲. تمرین چالشی

از یک لیست نام، دیکشنری بسازید که کلید نام و مقدار آن «طول نام > ۴» به‌صورت True/False باشد.

۱۶. چک‌لیست

- می‌توانم list comprehension بنویسم
- از enumerate و zip استفاده کرده‌ام

درس ۱۵ — کار با فایل متنی

۱. هدف یادگیری

فراگیر می‌تواند فایل متنی را بخواند و بنویسد، از with استفاده کند و توضیح دهد چرا مدیریت صحیح فایل مهم است.

۲. پیش‌نیاز

درس‌های قبلی

۳. زمان

۴۵ دقیقه

۴. انگیزه

داده‌ها نباید فقط در حافظه بمانند. لاگ، تنظیمات، خروجی گزارش و ذخیره‌سازی ساده همه با فایل شروع می‌شوند.

۵. واژه‌های کلیدی

- open / with
- Mode: r, w, a, x
- 'encoding='utf-۸
- read / readline / readlines / write

۷. کد نمونه

```
#نوشتن
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("خط اول\n")
    f.write("خط دوم\n")

#خواندن کل فایل
with open("notes.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

#خواندن خط‌به‌خط
with open("notes.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(">", line.strip())

#اضافه کردن به انتها
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("خط سوم\n")
```

۱۰. خطاهای رایج

```
FileNotFoundError: [Errno 2] No such file or directory: 'notes.txt'
```

فایل برای خواندن وجود ندارد. راه‌حل: مسیر را بررسی کنید یا اول با w بسازید.

- همیشه 'encoding='utf-۸ را برای فایل‌های فارسی مشخص کنید.

- استفاده از with تضمین می‌کند فایل حتی در صورت بروز خطا بسته شود.

۱۱. تمرین پایه

نام چند دوست را از کاربر بگیرید و در فایل friends.txt ذخیره کنید.

۱۲. تمرین چالشی

فایلی را بخوانید و تعداد خطوط و تعداد کلمات را گزارش دهید.

۱۶. چک‌لیست

- می‌توانم با with فایل بخوانم و بنویسم
- تفاوت r/w/a را می‌دانم
- utf-۸ را رعایت می‌کنم

درس ۱۶ — JSON

۱. هدف یادگیری

فراگیر می‌تواند داده را به JSON تبدیل کند، از JSON بخواند و کاربرد آن در ذخیره تنظیمات و تبادل داده را توضیح دهد.

۲. پیش‌نیاز

درس ۱۳ (دیکشنری) و درس ۱۵ (فایل)

۳. زمان

۴۰ دقیقه

۴. انگیزه

JSON زبان مشترک برنامه‌هاست: APIها، فایل تنظیمات، ذخیره‌سازی سبک. تقریباً هر جونیور دولوپر هر روز با JSON کار می‌کند.

۵. واژه‌های کلیدی

- JSON — JavaScript Object Notation
- json.dumps / json.loads
- json.dump / json.load
- Serialization — سریال‌سازی

۷. کد نمونه

```
import json

data = {
",سارا    "name": "
    "skills": ["python", "git"],
    "active": True,
    "score": 18.5
}

 JSONدیکشنری → رشته#
json_text = json.dumps(data, ensure_ascii=False, indent=2)
print(json_text)

ذخیره در فایل#
with open("user.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

خواندن از فایل#
```

```
with open("user.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded["name"])
print(loaded["skills"][0])
```

۸. خروجی

```
{
  "name": "سارا",
  "skills": ["python", "git"],
  "active": true,
  "score": 18.5
}
سارا
python
```

۱۰. خطاهای رایج

```
json.decoder.JSONDecodeError: Expecting value...
```

فایل خالی یا JSON نامعتبر است. راه‌حل: محتوا را بررسی کنید.

- ensure_ascii=False برای حفظ حروف فارسی ضروری است.

۱۱. تمرین پایه

یک دیکشنری از اطلاعات خودتان بسازید و در فایل me.json ذخیره کنید.

۱۲. تمرین چالشی

فایل JSON را بخوانید، یک فیلد را تغییر دهید و دوباره ذخیره کنید.

۱۶. چک‌لیست

- می‌توانم dumps/loads و dump/load را به کار ببرم
- ensure_ascii=False را بلدم
- می‌دانم JSON چه کاربردی دارد

درس ۱۷ — مدیریت خطا با try / except

۱. هدف یادگیری

فراگیر می‌تواند خطاهای قابل پیش‌بینی را با try/except مدیریت کند، از except چندگانه و else/finally استفاده کند و پیام خطای مناسب به کاربر نشان دهد.

۲. پیش‌نیاز

بخش ۱ و درس‌های فایل/تبدیل نوع

۳. زمان

۴۵ دقیقه

۴. انگیزه

برنامه حرفه‌ای نباید با اولین ورودی اشتباه کرش کند. مدیریت خطا تفاوت کد آموزشی و کد قابل اعتماد است.

۵. واژه‌های کلیدی

- try / except / else / finally
- Exception hierarchy
- ValueError / TypeError / FileNotFoundError / ZeroDivisionError
- raise — پرتاب عمدی خطا

۷. کد نمونه

```
def safe_divide(a: str, b: str) -> float | None:
    try:
        x = float(a)
        y = float(b)
        result = x / y
    except ValueError:
."(ورودی باید عدد باشد: خطا        print("
        return None
    except ZeroDivisionError:
."(تقسیم بر صفر مجاز نیست: خطا        print("
        return None
    else:
."(محاسبه موفق بود)        print("
        return result
    finally:
."(پایان تلاش برای محاسبه)        print("
```

```
print(safe_divide("10", "2"))
print(safe_divide("10", "0"))
print(safe_divide("abc", "2"))
```

۸. خروجی

```
محاسبه موفق بود.
پایان تلاش برای محاسبه.
5.0
خطا :تقسیم بر صفر مجاز نیست.
پایان تلاش برای محاسبه.
None
خطا :ورودی باید عدد باشد.
پایان تلاش برای محاسبه.
None
```

۱۰. خطاهای رایج

گرفتن Exception به‌صورت کلی و پنهان کردن خطا (بد است). تا حد ممکن except را مشخص بنویسید.

فراموش کردن بازگرداندن مقدار در مسیر except.

۱۱. تمرین پایه

برنامه‌ای بنویسید که سن را بگیرد و اگر عدد نبود پیام مناسب بدهد.

۱۲. تمرین چالشی

تابعی بنویسید که نام فایل را بگیرد، محتوا را بخواند و اگر فایل نبود پیام بدهد؛ در غیر این صورت تعداد خطوط را برگرداند.

۱۳. پاسخ پایه

```
try:
    age = int(input("سن: "))
    print(f"سن شما {age} است.")
except ValueError:
    print("لطفاً فقط عدد وارد کنید.")
```

۱۶. چک‌لیست

- ☐ می‌توانم try/except بنویسم
- ☐ چند نوع خطا را جدا مدیریت می‌کنم
- ☐ از finally در جای مناسب استفاده می‌کنم

پروژه فصل ۲ — مدیر وظایف (To-Do) با ذخیره JSON

صورت مسئله

یک برنامه خط فرمان برای مدیریت کارها بنویسید که امکانات زیر را داشته باشد:

- افزودن کار جدید
- نمایش لیست کارها
- علامت‌زدن کار به‌عنوان انجام‌شده
- حذف کار
- ذخیره و بارگذاری خودکار از فایل tasks.json

راهنمای طراحی

- هر کار یک دیکشنری باشد: id, title, done
- کل داده‌ها یک لیست از دیکشنری‌ها
- توابع جدا برای load / save / add / list / complete / delete
- منوی متنی در حلقه while

کد اسکلت پیشنهادی

```
import json
from pathlib import Path

DATA_FILE = Path("tasks.json")

def load_tasks() -> list[dict]:
    if not DATA_FILE.exists():
        return []
    with open(DATA_FILE, "r", encoding="utf-8") as f:
        return json.load(f)

def save_tasks(tasks: list[dict]) -> None:
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(tasks, f, ensure_ascii=False, indent=2)

def add_task(tasks: list[dict], title: str) -> None:
    new_id = max((t["id"] for t in tasks), default=0) + 1
    tasks.append({"id": new_id, "title": title, "done": False})

def show_tasks(tasks: list[dict]) -> None:
    if not tasks:
."(لیست خالی است        print("
        return
    for t in tasks:
        status = "✓" if t["done"] else "o"
        print(f"{t['id']}. [{status}] {t['title']}")
```

#بقیه توابع + منوی اصلی را خودتان کامل کنید

روبریک ارزیابی

- ذخیره و بارگذاری JSON بدون خطا (۲۵٪)
- عملیات افزودن / نمایش / تکمیل / حذف (۳۰٪)
- مدیریت ورودی و خطاها (۲۰٪)
- ساختار تابعی تمیز و خوانا (۱۵٪)
- تجربه کاربری قابل قبول (۱۰٪)

آزمون پایان بخش ۲ (۱۰ سؤال)

۱. فرق اصلی لیست و تاپل چیست؟

۲. چطور عنصری به انتهای لیست اضافه کنیم؟

۳. خروجی {۱, ۲, ۲, ۳} چیست؟

۴. برای دسترسی امن به کلید دیکشنری از چه متدی استفاده می‌کنیم؟

۵. یک list comprehension بنویسید که مربع اعداد زوج ۱ تا ۱۰ را بسازد.

۶. چرا از with open استفاده می‌کنیم؟

۷. تفاوت json.dump و json.dumps چیست؟

۸. ensure_ascii=False چه کاربردی دارد؟

۹. چه زمانی از except ValueError استفاده می‌کنیم؟

۱۰. finally چه زمانی اجرا می‌شود؟

پاسخنامه

۱. تاپل تغییرناپذیر است، لیست تغییرپذیر.

۲. append

۳. {۱, ۲, ۳} — اعضای تکراری حذف می‌شوند.

۴. get

۵. [n**۲ for n in range(۱,۱۱) if n٪۲==۰]

۶. تا فایل حتی در صورت خطا به‌طور خودکار بسته شود.

۷. dump روی فایل می‌نویسد، dumps رشته برمی‌گرداند.

۸. حفظ کاراکترهای غیرASCII مثل فارسی.

۹. وقتی تبدیل نوع یا مقدار ورودی نامعتبر است.

۱۰. در هر صورت؛ چه خطا رخ بدهد چه ندهد.

خلاصه بخش ۲ و مهارت نهایی

در این بخش ساختمان‌داده‌های اصلی را یاد گرفتید، داده را در فایل و JSON ذخیره و بازیابی کردید و خطاها را به‌صورت کنترل‌شده مدیریت کردید. پروژه To-Do شما را یک قدم به برنامه‌های واقعی نزدیک کرد.

مهارت نهایی مورد انتظار

- ☐ می‌توانم لیست، تاپل، ست و دیکشنری را درست انتخاب و استفاده کنم
- ☐ می‌توانم comprehension ساده بنویسم
- ☐ می‌توانم فایل متنی و JSON بخوانم و بنویسم
- ☐ می‌توانم با try/except خطاهای رایج را مدیریت کنم
- ☐ می‌توانم یک برنامه کوچک پایدار با ذخیره داده بسازم

☐ بخش ۲ به پایان رسید. حالا آماده‌اید وارد بخش ۳ شوید: برنامه‌نویسی حرفه‌ای (PEP۸، توابع پیشرفته، OOP، تست و گیت).

☐ برای دریافت بخش ۳ بگویید: ادامه بده

AHRIVAN | اهریمن



بخش ۳

برنامه‌نویسی حرفه‌ای

PEP ۸ • توابع پیشرفته • شیءگرایی
تست • دیباگینگ • گیت و گیتهاب

پایتون حرفه‌ای — از صفر تا جونیور دولوپر

AHRIVAN | اهریمن

نقشه بخش ۳ و اهداف کلی

در پایان این بخش، فراگیر باید بتواند کد تمیز و مطابق PEP۸ بنویسد، از قابلیت‌های پیشرفته توابع استفاده کند، کلاس و وراثت را پیاده‌سازی کند، تست واحد بنویسد، لاگ ثبت کند و با گیت در سطح پایه کار کند.

- درس ۱۸: PEP ۸، نام‌گذاری و تمیزی کد
- درس ۱۹: توابع پیشرفته (*args، **kwargs، lambda، map/filter)
- درس ۲۰: شیءگرایی — کلاس و شیء
- درس ۲۱: وراثت و متدهای ویژه
- درس ۲۲: ساختار پروژه و ماژول‌ها
- درس ۲۳: تست با pytest
- درس ۲۴: دیباگینگ و logging
- درس ۲۵: مقدمه گیت و گیتهاب
- پروژه فصل: کتابخانه مدیریت کتاب با کلاس + تست

درس ۱۸ — PEP ۸، نام‌گذاری و تمیزی کد

۱. هدف یادگیری

فراگیر می‌تواند اصول اصلی PEP ۸ را رعایت کند، نام‌های معنادار انتخاب کند و کد خوانا بنویسد.

۲. پیش‌نیاز

بخش ۱ و ۲

۳. زمان

۴۰ دقیقه

۴. انگیزه

در تیم واقعی، کد بیشتر خوانده می‌شود تا نوشته شود. کدی که فقط «کار می‌کند» ولی خوانا نیست، هزینه نگهداری بالایی دارد. PEP ۸ استاندارد رسمی استایل پایتون است.

۵. واژه‌های کلیدی

- PEP ۸ — راهنمای سبک رسمی پایتون
- snake_case / PascalCase
- Readability — خوانایی
- linter (مثل ruff / flake۸)

۷. اصول کلیدی + نمونه

```
#بد
def f(x,y):
  return x+y


#خوب – مطابق PEP 8
def calculate_sum(first_number: float, second_number: float) -> float:
    """جمع دو عدد را برمی‌گرداند."""
    return first_number + second_number


#نام‌گذاری
user_name = "sara"          # snake_case: متغیر و تابع
MAX_RETRY = 3               # UPPER_SNAKE_CASE: ثابت
class UserProfile:          # PascalCase: کلاس
    pass
```

۱۰. نکات مهم

- تورفتگی: ۴ فاصله (از Tab خالص اجتناب کنید)
- طول خط: ترجیحاً حداکثر ۷۹–۸۸ کاراکتر
- فاصله اطراف عملگرها و بعد از کاما
- importها در بالای فایل و گروه‌بندی‌شده

☐ ابزار پیشنهادی: افزونه Ruff یا Black در VS Code برای فرمت خودکار.

۱۱. تمرین پایه

یک تابع نامرتب را مطابق PEP ۸ بازنویسی کنید (نام، فاصله، docstring).

۱۲. تمرین چالشی

یک فایل ۱۰–۱۵ خطی از کد قبلی خودتان را با رعایت کامل PEP ۸ اصلاح کنید.

۱۶. چک‌لیست

- ☐ snake_case و PascalCase را درست استفاده می‌کنم
- ☐ docstring ساده می‌نویسم
- ☐ تورفتگی و فاصله‌ها را رعایت می‌کنم

درس ۱۹ — توابع پیشرفته

۱. هدف یادگیری

فراگیر می‌تواند از *args و **kwargs استفاده کند، تابع ناشناس (lambda) بنویسد و map/filter را در موارد مناسب به کار ببرد.

۲. پیش‌نیاز

درس ۸ (توابع)

۳. زمان

۵۰ دقیقه

۴. انگیزه

در کتابخانه‌ها و فریم‌ورک‌ها مدام با تعداد آرگومان متغیر و توابع مرتبه بالاتر روبه‌رو می‌شوید. این ابزارها کد را انعطاف‌پذیرتر می‌کنند.

۵. واژه‌های کلیدی

- *args — آرگومان‌های موقعیتی متغیر
- **kwargs — آرگومان‌های کلیدواژه‌ای متغیر
- lambda — تابع ناشناس
- map / filter

۷. کد نمونه

```
def print_all(*args):
    for item in args:
        print(item)


print_all("a", "b", "c")


def build_profile(**kwargs):
    return kwargs


")تهران", age=17, city="سارا"profile = build_profile(name="
print(profile)


# lambda + map/filter
numbers = [1, 2, 3, 4, 5, 6]
squares = list(map(lambda n: n ** 2, numbers))
evens = list(filter(lambda n: n % 2 == 0, numbers))
print(squares)
print(evens)
```

```
#در عمل اغلب comprehension خواناتر است:
squares2 = [n ** 2 for n in numbers]
```

۸. خروجی

```
a
b
c
'}تهران', 'age': 17, 'city': 'سارا{'name': '
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
```

۱۰. خطاهای رایج

قرار دادن آرگومان معمولی بعد از *args بدون نام → SyntaxError.

استفاده بیش از حد از lambda برای منطق پیچیده (خوانایی کم می‌شود).

۱۱. تمرین پایه

تابعی بنویسید که هر تعداد عدد بگیرد و مجموع را برگرداند (*args).

۱۲. تمرین چالشی

با filter و lambda فقط رشته‌های طولانی‌تر از ۳ کاراکتر را از یک لیست جدا کنید.

۱۶. چک‌لیست

- ☐ می‌توانم *args و **kwargs بنویسم
- ☐ lambda ساده می‌نویسم
- ☐ فرق map/filter با comprehension را می‌دانم

درس ۲۰ — شیءگرایی: کلاس و شیء

۱. هدف یادگیری

فراگیر می‌تواند کلاس تعریف کند، شیء بسازد، ویژگی و متد اضافه کند و تفاوت کلاس و شیء را توضیح دهد.

۲. پیش‌نیاز

توابع و دیکشنری

۳. زمان

۵۰ دقیقه

۴. انگیزه

وقتی داده و رفتار مربوط به یک مفهوم با هم جمع می‌شوند (کاربر، سفارش، محصول)، کلاس مدل طبیعی‌تری نسبت به دیکشنری پراکنده است.

۵. واژه‌های کلیدی

- Class — قالب
- Object / Instance — نمونه
- __init__ — سازنده
- self — ارجاع به خود شیء
- Attribute / Method

۷. کد نمونه

```
class Book:
    def __init__(self, title: str, author: str, pages: int):
        self.title = title
        self.author = author
        self.pages = pages
        self.is_read = False

    def mark_as_read(self) -> None:
        self.is_read = True

    def summary(self) -> str:
"خواندنشده" if self.is_read else "خواندهشده        status = "
) [{status}]"صفحه        return f"{self.title} – {self.author} ({self.pages}

", 220)علی رضایی", "پایتون آسان"b1 = Book("
print(b1.summary())
```

```
b1.mark_as_read()
print(b1.summary())
```

۸. خروجی

```
پایتون آسان – علی رضایی (220 صفحه) [خواندنشده]
پایتون آسان – علی رضایی (220 صفحه) [خواندشده]
```

۱۰. خطاهای رایج

```
TypeError: Book() takes no arguments
```

فراموش کردن self در __init__ یا فراخوانی اشتباه.

```
NameError: name 'self' is not defined
```

self فقط داخل متدهای کلاس معنا دارد.

۱۱. تمرین پایه

کلاس Student با نام، سن و متد introduce بسازید.

۱۲. تمرین چالشی

کلاس BankAccount با deposit و withdraw و جلوگیری از موجودی منفی.

۱۶. چک‌لیست

- ☐ می‌توانم کلاس و __init__ بنویسم
- ☐ متد تعریف می‌کنم و فراخوانی می‌کنم
- ☐ تفاوت کلاس و شیء را می‌فهمم

درس ۲۱ — وراثت و متدهای ویژه

۱. هدف یادگیری

فراگیر می‌تواند از وراثت استفاده کند، متد را override کند و متدهای ویژه مثل __str__ را پیاده کند.

۲. پیش‌نیاز

درس ۲۰

۳. زمان

۵۰ دقیقه

۴. انگیزه

وراثت از تکرار جلوگیری می‌کند و سلسله‌مراتب طبیعی مفاهیم را مدل می‌کند (مثلاً Vehicle → Car / Bike).

۵. واژه‌های کلیدی

- Inheritance — وراثت
- Override — بازنویسی متد
- super()
- __str__ / __repr__

۷. کد نمونه

```
class Animal:
    def __init__(self, name: str):
        self.name = name

    def speak(self) -> str:
        return "..."

    def __str__(self) -> str:
        return f"Animal({self.name})"

class Dog(Animal):
    def speak(self) -> str:
!"واق واق"        return "

class Cat(Animal):
    def speak(self) -> str:
!"میو"        return "
```

```
")ریکسdog = Dog("
")پشمکcat = Cat("
print(dog, "→", dog.speak())
print(cat, "→", cat.speak())
```

۸. خروجی

```
)Animalریکس → (واق واق!
)Animalپشمک → (میو!
```

۱۱. تمرین پایه

کلاس Vehicle و دو کلاس فرزند Car و Bike با متد move متفاوت.

۱۲. تمرین چالشی

در کلاس Book متد __str__ را طوری بنویسید که خروجی خوانا بدهد و کلاس EBook از آن ارث ببرد و فیلد file_size اضافه کند.

۱۶. چک‌لیست

- ☐ می‌توانم کلاس فرزند بسازم
- ☐ متد را override می‌کنم
- ☐ __str__ می‌نویسم

درس ۲۲ — ساختار پروژه و ماژول‌ها

۱. هدف یادگیری

فراگیر می‌تواند یک پروژه چندفایلی ساده سازماندهی کند، import نسبی/مطلق را درست بنویسد و از '__main__ == __name__ if__' استفاده کند.

۲. پیش‌نیاز

ماژول‌ها (درس ۱۰) و توابع/کلاس‌ها

۳. زمان

۴۰ دقیقه

۴. انگیزه

پروژه‌های واقعی یک فایل نیستند. ساختار درست، کار تیمی و توسعه بعدی را ممکن می‌کند.

۵. واژه‌های کلیدی

- Package / Module
- __init__.py
- '__main__ == __name__ if__'
- Import path

۷. ساختار پیشنهادی

```
my_project/
├── main.py
├── models.py          # کلاس‌ها
├── services.py        # منطق کسب‌وکار
├── utils.py           # توابع کمکی
└── data/
    └── tasks.json
# models.py
class Task:
    def __init__(self, title: str):
        self.title = title
        self.done = False


# main.py
from models import Task


def main():
    t = Task("یادگیری گیت")
    print(t.title)
```

```
if __name__ == "__main__":
    main()
```

۱۶. چک‌لیست

- [ ] پروژه را به چند فایل منطقی تقسیم می‌کنم
- [ ] از 'if __name__ == '__main__' استفاده می‌کنم
- [ ] import تمیز می‌نویسم

درس ۲۳ — تست با pytest

۱. هدف یادگیری

فراگیر می‌تواند تست واحد ساده با pytest بنویسد، اجرا کند و اهمیت تست را توضیح دهد.

۲. پیش‌نیاز

توابع و کلاس‌ها

۳. زمان

۴۵ دقیقه

۴. انگیزه

بدون تست، هر تغییر ممکن است چیزی را بشکند. تست یعنی اعتماد به کد و امکان بازآرایی (refactor) امن.

۵. واژه‌های کلیدی

- Unit Test — تست واحد
- pytest
- assert
- Test discovery

۷. کد نمونه

```
# calculator.py
def add(a: float, b: float) -> float:
    return a + b


def divide(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b


# test_calculator.py
import pytest
from calculator import add, divide


def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0


def test_divide():
    assert divide(10, 2) == 5
```

```
def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)
```

اجرا: در ترمینال بنویسید pytest -q

۱۰. خطاهای رایج

نام فایل یا تابع تست با test_ شروع نشده → pytest پیدا نمی‌کند.

۱۱. تمرین پایه

برای تابع maximum دو تست بنویسید.

۱۲. تمرین چالشی

برای کلاس BankAccount تست واریز و برداشت بنویسید.

۱۶. چک‌لیست

- ☐ می‌توانم فایل تست بنویسم
- ☐ از assert و pytest.raises استفاده می‌کنم
- ☐ pytest را اجرا کرده‌ام

درس ۲۴ — دیباگینگ و logging

۱. هدف یادگیری

فراگیر می‌تواند با چاپ هدفمند و ماژول logging مشکل را پیدا کند و سطح لاگ مناسب انتخاب کند.

۲. پیش‌نیاز

مدیریت خطا

۳. زمان

۴۰ دقیقه

۴. انگیزه

print برای دیباگ سریع خوب است، اما در برنامه واقعی logging جایگزین حرفه‌ای آن است: سطح‌بندی، زمان‌دار بودن و امکان خاموش کردن.

۵. واژه‌های کلیدی

- Debugging
- logging (DEBUG, INFO, WARNING, ERROR)
- Breakpoint (در VS Code)

۷. کد نمونه

```
import logging


logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s | %(levelname)s | %(message)s"
)


def process_order(quantity: int) -> None:
")شروع پردازش سفارش    logging.info("
    if quantity <= 0:
: %s", quantity)تعداد نامعتبر        logging.error("
        return
: %s", quantity)تعداد    logging.debug("
")سفارش با موفقیت ثبت شد    logging.info("


process_order(3)
process_order(0)
```

۱۶. چک‌لیست

- ☐ از logging به‌جای print در منطق اصلی استفاده می‌کنم
- ☐ سطح لاگ مناسب انتخاب می‌کنم
- ☐ می‌توانم در VS Code breakpoint بگذارم

درس ۲۵ — مقدمه گیت و گیتهاب

۱. هدف یادگیری

فراگیر می‌تواند مخزن بسازد، تغییرات را commit کند، به گیتهاب push کند و مفهوم branch را در سطح پایه توضیح دهد.

۲. پیش‌نیاز

کار با ترمینال

۳. زمان

۵۰ دقیقه

۴. انگیزه

بدون کنترل نسخه، همکاری و بازگشت به نسخه‌های قبلی practically غیرممکن است. گیت استاندارد صنعت است.

۵. واژه‌های کلیدی

- Repository — مخزن
- commit / push / pull
- branch / merge
- GitHub

۷. دستورات پایه

```
git init
git status
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/USER/REPO.git
git push -u origin main


#کار روزمره
git pull
git checkout -b feature/login
git add .
git commit -m "Add login form"
git push -u origin feature/login
```

پیام commit را واضح و به انگلیسی یا فارسی منسجم بنویسید. هر commit یک واحد منطقی باشد.

۱۱. تمرین پایه

یک پوشه پروژه را git init کنید، فایل اضافه کنید و اولین commit را بزنید.

۱۲. تمرین چالشی

مخزن را روی گیتهاب بسازید، push کنید و یک branch جدید برای یک ویژگی کوچک ایجاد کنید.

۱۶. چک‌لیست

- ☐ git init / add / commit را بلدم
- ☐ روی گیتهاب مخزن ساخته‌ام
- ☐ مفهوم branch را می‌فهمم

پروژه فصل ۳ — کتابخانه مدیریت کتاب (با کلاس + تست)

صورت مسئله

سیستم ساده‌ای برای مدیریت کتاب‌ها بنویسید که شامل موارد زیر باشد:

- کلاس Book با عنوان، نویسنده، وضعیت امانت
- کلاس Library برای افزودن، امانت دادن، برگرداندن و لیست کردن کتاب‌ها
- ذخیره و بارگذاری داده در JSON
- حداقل ۳ تست pytest برای منطق اصلی
- ساختار چندفایلی (models.py, library.py, main.py, test_library.py)

روبریک ارزیابی

- مدل شیءگرا و تمیز (۲۵٪)
- عملیات اصلی Library (۲۵٪)
- ذخیره/بارگذاری JSON (۱۵٪)
- تست‌های معنادار (۲۰٪)
- ساختار پروژه و PEP۸ (۱۵٪)

آزمون پایان بخش ۳ (۱۰ سؤال)

۱. نام متغیرها در PEP ۸ معمولاً با چه سبکی نوشته می‌شود؟

۲. *args چه چیزی را دریافت می‌کند؟

۳. self در متد کلاس به چه چیزی اشاره دارد؟

۴. متد سازنده کلاس چه نامی دارد؟

۵. برای بازنویسی رفتار چاپ شیء از چه متدی استفاده می‌کنیم؟

۶. چرا '__main__' == __name__ if' مهم است؟

۷. برای اجرای تست‌ها چه دستوری رایج است؟

۸. سطح لاگ مناسب برای خطای جدی چیست؟

۹. دستور ثبت تغییرات در گیت چیست؟

۱۰. یک مزیت اصلی نوشتن تست واحد را بنویسید.

پاسخنامه

۱. snake_case

۲. تعداد متغیر آرگومان موقعیتی

۳. خود شیء

۴. __init__

۵. __str__

۶. تا کد فقط هنگام اجرای مستقیم فایل اجرا شود نه با import

۷. pytest

۸. ERROR

۹. git commit

۱۰. اطمینان از صحت رفتار و امکان refactor امن

خلاصه بخش ۳ و مهارت نهایی

در این بخش از کدنویسی «کار می‌کند» به کدنویسی «حرفه‌ای و قابل نگهداری» رسیدید: استایل، توابع پیشرفته، شیءگرایی، تست، لاگ و کنترل نسخه.

مهارت نهایی مورد انتظار

- ☐ کد را مطابق PEP ۸ می‌نویسم
- ☐ از *args/kwargs و lambda در جای مناسب استفاده می‌کنم
- ☐ کلاس، شیء و وراثت ساده پیاده می‌کنم
- ☐ تست واحد با pytest می‌نویسم
- ☐ با logging و گیت در سطح پایه کار می‌کنم

☐ بخش ۳ به پایان رسید. اکنون آماده‌اید وارد بخش ۴ شوید: پروژه‌های واقعی و نزدیک به بازار کار.

☐ برای دریافت بخش ۴ (پروژه‌های واقعی + پروژه پایانی) بگویید: ادامه بده

AHRIVAN | اهریمن



بخش ۴ و ۵

پروژه‌های واقعی + سنجش نهایی

CLI • To-Do • وب‌اسکرپر • پروژه پایانی

روبریک • چک‌لیست مهارت • مسیر ادامه

پایتون حرفه‌ای — از صفر تا جونیور دولوپر

AHRIVAN | اهریمن

بخش ۴ — پروژه‌های واقعی

در این بخش چند پروژه کاربردی می‌سازید که مهارت‌های بخش‌های قبلی را با هم ترکیب می‌کنند. هدف این است که از «تمرین درسی» به «برنامه قابل استفاده» برسید.

پروژه ۱ — ماشین‌حساب CLI پیشرفته

صورت مسئله

ماشین‌حسابی بنویسید که از تاریخچه محاسبات پشتیبانی کند، نتیجه آخر را در حافظه نگه دارد و امکان پاک کردن حافظه داشته باشد.

ویژگی‌های الزامی

- عملیات + - * / و توان
- دستور history برای نمایش تاریخچه
- دستور clear برای پاک کردن تاریخچه
- مدیریت تقسیم بر صفر و ورودی نامعتبر
- خروج با exit

راهنمای پیاده‌سازی

```
#ساختار پیشنهادی
def calculate(op, a, b): ...
def show_history(history): ...
def main():
    history = []
    while True:
        cmd = input("> ").strip()
پردازش دستورات        #
```

روبریک (۲۰ امتیاز)

- صحت محاسبات (۶)
- تاریخچه و حافظه (۵)
- مدیریت خطا (۵)
- خوانایی و ساختار (۴)

پروژه ۲ — مدیر وظایف کامل (To-Do Pro)

صورت مسئله

نسخه کامل‌تر پروژه فصل ۲ با اولویت، تاریخ سررسید و فیلتر.

ویژگی‌های الزامی

- افزودن کار با عنوان، اولویت (low/medium/high) و تاریخ اختیاری
- نمایش همه / فقط انجام‌نشده / بر اساس اولویت
- تکمیل و حذف کار
- ذخیره خودکار در tasks.json
- ساختار چندفایلی (models, storage, cli)

نمونه مدل داده

```
{
  "id": 1,
",تمرین گیت  "title": "
  "priority": "high",
  "done": false,
  "due": "2026-11-01"
}
```

روبریک (۲۵ امتیاز)

- مدل داده و ذخیره JSON (۷)
- عملیات CRUD کامل (۸)
- فیلتر و اولویت (۵)
- ساختار پروژه و UX (۵)

پروژه ۳ — دفترچه تلفن با جستجو

صورت مسئله

دفترچه تلفن خط فرمان با ذخیره JSON، جستجوی نام و ویرایش شماره.

- افزودن مخاطب (نام + شماره)
- جستجو بر اساس بخشی از نام
- ویرایش و حذف
- نمایش مرتب‌شده بر اساس نام

روبریک (۱۵ امتیاز)

- ذخیره و بارگذاری (۴)
- جستجو و ویرایش (۶)
- ساختار تمیز (۵)

پروژه ۴ — وب‌اسکرپر ساده با requests

صورت مسئله

برنامه‌ای بنویسید که عنوان و وضعیت کد HTTP چند آدرس را بگیرد و در فایل ذخیره کند.

کتابخانه requests را با pip install requests نصب کنید. فقط برای یادگیری و رعایت robots.txt و شرایط استفاده سایت‌ها.

کد اسکلت

```
import requests
from pathlib import Path
import json

urls = [
    "https://www.example.com",
    "https://httpbin.org/status/404",
]

results = []
for url in urls:
    try:
        r = requests.get(url, timeout=5)
        results.append({
            "url": url,
            "status": r.status_code,
            "ok": r.ok
        })
    except requests.RequestException as e:
        results.append({"url": url, "error": str(e)})

Path("results.json").write_text(
    json.dumps(results, ensure_ascii=False, indent=2),
    encoding="utf-8"
)
:", results)ذخیره شدprint("
```

تمرین توسعه‌ای

عنوان صفحه (تگ title) را هم استخراج کنید. برای شروع می‌توانید از r.text و جستجوی ساده رشته استفاده کنید یا با BeautifulSoup (اختیاری).

روبریک (۱۵ امتیاز)

- درخواست موفق و مدیریت خطا (۶)
- ذخیره JSON (۴)
- کد تمیز و قابل گسترش (۵)

پروژه پایانی — سیستم مدیریت کتابخانه شخصی

هدف پروژه

یک برنامه کامل CLI برای مدیریت کتاب‌های شخصی بسازید که نشان دهد می‌توانید یک مسئله واقعی را از صفر تا محصول کوچک طراحی و پیاده‌سازی کنید.

قابلیت‌های الزامی

- افزودن کتاب (عنوان، نویسنده، ژانر، سال)
- لیست همه کتاب‌ها
- جستجو بر اساس عنوان یا نویسنده
- علامت‌زدن به‌عنوان خوانده‌شده / خوانده‌نشده
- حذف کتاب
- آمار: تعداد کل، تعداد خوانده‌شده، ژانر محبوب
- ذخیره و بارگذاری خودکار JSON
- ساختار چندفایلی
- حداقل ۴ تست pytest
- فایل README.md با توضیح نصب و اجرا

ساختار پیشنهادی پروژه

```
library_app/
├── main.py
 Bookکلاس├── models.py          #
├── storage.py         # load/save JSON
 Libraryمنطق├── services.py        #
منو و تعامل کاربر├── cli.py             #
├── tests/
│   └── test_library.py
├── data/
│   └── books.json
└── README.md
```

نمونه کلاس Book

```
from dataclasses import dataclass, asdict


@dataclass
class Book:
    id: int
    title: str
    author: str
    genre: str
    year: int
    is_read: bool = False
```

```
def to_dict(self) -> dict:
    return asdict(self)
```

روبریک ارزیابی پروژه پایانی (۱۰۰ امتیاز)

- تحلیل و طراحی (مدل داده + ساختار پوشه) — ۱۵
- پیاده‌سازی قابلیت‌های اصلی — ۳۰
- ذخیره/بارگذاری و پایداری داده — ۱۵
- تست‌های واحد معنادار — ۱۵
- کیفیت کد (PEP۸، نام‌گذاری، توابع کوچک) — ۱۵
- README و تجربه کاربری — ۱۰

معیار عالی بودن

پروژه‌ای عالی است که بدون توضیح شفاهی قابل فهم باشد، خطاهای ورودی را تحمل کند، تست‌هایش سبز باشند و README آن در کمتر از ۲ دقیقه کسی را به اجرای برنامه برساند.

بخش ۵ — سنجش نهایی و مسیر ادامه

۵٫۱ چک‌لیست مهارت نهایی دوره

اگر موارد زیر را تیک می‌زنید، به سطح جونیور ابتدایی نزدیک شده‌اید:

مبانی

- ☐ محیط کار (Python + VS Code) را مستقل راه‌اندازی می‌کنم
- ☐ متغیر، نوع داده، شرط، حلقه و تابع را روان می‌نویسم
- ☐ خطاهای رایج را از روی پیام پایتون تشخیص می‌دهم

داده و فایل

- ☐ لیست، دیکشنری و کامپرهنشن را درست انتخاب می‌کنم
- ☐ فایل متنی و JSON می‌خوانم و می‌نویسم
- ☐ try/except را برای ورودی و فایل به کار می‌برم

حرفه‌ای‌سازی

- ☐ PEP ۸ را در حد معقول رعایت می‌کنم
- ☐ کلاس و وراثت ساده می‌نویسم
- ☐ تست واحد با pytest می‌نویسم و اجرا می‌کنم
- ☐ با گیت commit و push می‌کنم

پروژه

- ☐ یک برنامه CLI چندفایلی از صفر طراحی و پیاده کرده‌ام
- ☐ README نوشته‌ام و پروژه‌ام روی گیتهاب است

۵٫۲ آزمون جامع نهایی (۲۰ سؤال — نمونه)

۱. فرق list و tuple چیست؟

۲. خروجی list(range(۳)) چیست؟

۳. چرا input همیشه str برمی‌گرداند؟

۴. f-string چه مزیتی نسبت به concatenation دارد؟

۵. IndentationError معمولاً چه زمانی رخ می‌دهد؟

۶. *args چه کاربردی دارد؟

۷. متد __init__ چه نقشی دارد؟

۸. فرق json.load و json.loads چیست؟

۹. ensure_ascii=False چرا مهم است؟

۱۰. pytest چگونه فایل‌های تست را پیدا می‌کند؟

۱۱. سطح لاگ WARNING چه معنایی دارد؟

۱۲. دستور ایجاد branch جدید در گیت چیست؟

۱۳. یک مزیت dataclass نسبت به کلاس معمولی؟

۱۴. KeyError را چطور به‌صورت امن مدیریت می‌کنیم؟

۱۵. اصل DRY یعنی چه؟

۱۶. چه زمانی از set استفاده می‌کنیم؟

۱۷. 'if __name__ == '__main__' از چه مشکلی جلوگیری می‌کند؟

۱۸. map و list comprehension از نظر خوانایی معمولاً کدام ترجیح داده می‌شود؟

۱۹. یک تست خوب چه ویژگی‌ای دارد؟

۲۰. برای ورود به حوزه وب بعد از این دوره چه مسیری پیشنهاد می‌شود؟

پاسخنامه خلاصه

۱. tuple تغییرناپذیر است. ۲. [۰،۱،۲]. ۳. ورودی کاربر متن است. ۴. خوانایی و فرمت‌بندی آسان. ۵. تورفتگی اشتباه بعد از :. ۶. آرگومان موقعیتی متغیر. ۷. مقداردهی اولیه شیء. ۸. load از فایل، loads از رشته. ۹. حفظ فارسی. ۱۰. نام test_*. هشدار غیربحرانی. ۱۲. git checkout -b. کد کمتر و خواناتر. ۱۴. با get یا try. ۱۵. تکرار نکن. ۱۶. یکتایی و عضویت. ۱۷. اجرای ناخواسته هنگام import. معمولاً ۱۹. comprehension. مستقل، سریع، واضح. ۲۰. Flask یا Django به همراه HTML/CSS پایه و کار با API.

۵٫۳ مسیرهای ادامه بعد از این دوره

مسیر وب (Backend)

- HTML/CSS پایه + HTTP
- Flask یا FastAPI (پیشنهاد برای شروع سریع) یا Django
- پایگاه‌داده SQLite/PostgreSQL و ORM
- احراز هویت و REST API

مسیر داده

- NumPy و Pandas
- تجسم با Matplotlib / Seaborn
- SQL
- پروژه تحلیل داده واقعی

مسیر هوش مصنوعی / یادگیری ماشین

- ریاضی پایه (جبر خطی، آمار)
- Scikit-learn
- آشنایی با PyTorch یا TensorFlow
- پروژه‌های کلاسیک (طبقه‌بندی، رگرسیون)

مسیر اتوماسیون و اسکریپت‌نویسی

- کار با فایل و پوشه‌ها (pathlib)
- زمان‌بندی کارها
- اتوماسیون مرورگر (Playwright/Selenium) در حد نیاز
- ربات‌های ساده و گزارش‌گیری

۵٫۴ توصیه‌های ورود به بازار کار / کارآموزی

- ۳ تا ۵ پروژه خوب روی گیتهاب با README قوی داشته باشید
- کد تمیز مهم‌تر از تعداد پروژه‌های ناتمام است
- یک پروژه را عمیق کنید (تست، ساختار، مستندات)
- در رزومه روی «چه مسئله‌ای حل کردید» تمرکز کنید نه فقط لیست تکنولوژی
- برای جونیور: تسلط به پایتون + گیت + یک حوزه (وب یا داده) کافی است تا مصاحبه بگیرید

تبریک! شما مسیر صفر تا آمادگی جونیور را در این جزوه طی کردید. از اینجا به بعد، با ساختن پروژه‌های واقعی و بازخورد گرفتن، رشد اصلی شروع می‌شود.

مجموعه کامل این دوره در چهار فایل ارائه شد: بخش ۰+۱، بخش ۲، بخش ۳، و بخش ۴+۵. می‌توانید آن‌ها را کنار هم به‌عنوان جزوه جامع استفاده کنید.

پایان دوره — پایتون حرفه‌ای از صفر تا جونیور دولوپر

پایتون رو ادامه بده و از خطا نترس

ارتباط با ما: @AhR1VaN

AHRIVAN | اهریمن